A Declarative Specification for Clinical Reference Range Number Line Visualizations

A Major Qualifying Project (MQP) Report
Submitted to the Faculty of
WORCESTER POLYTECHNIC INSTITUTE
in partial fulfillment of the requirements
for the Degree of Bachelor of Science in

Computer Science,
Bioinformatics & Computational Biology

By:

Zoë Fisk
Skyler Lin
Zachary Medailleu
Morgan Vasiliou

Project Advisor:

Lane Harrison

Date: April 28th 2026

This report represents work of WPI undergraduate students submitted to the faculty as evidence of a degree requirement. WPI routinely publishes these reports on its website without editorial or peer review. For more information about the projects program at WPI, see http://www.wpi.edu/Academics/Projects.

Abstract

For patients with chronic diseases, reference range number lines (RRNLs) can be useful tools in monitoring their health. Existing studies on RRNLs explore how design influences the patient's understanding of health status, risk, and resulting behaviors. Despite their usefulness in healthcare contexts, there are few visualization tools for creating RRNLs. In this paper, we introduce a new domain-specific language (DSL) that streamlines the creation of reference range number lines. This DSL is based on examples of RRNLs in literature, and supports the reproduction of these graphs with the input of a few data points, balancing usability and customization. Future iterations may support more expansive customization, including visual embellishments.

Acknowledgements

Thank you to our advisor, Professor Lane Harrison, for his guidance and insight throughout this project. We are incredibly appreciative of your continued encouragement and dedication to this project.

The authors wish to thank WPI Data Science PhD student and visualization literacy researcher Karen Bonilla Janssens (advised by Professor Lane Harrison) for advice and guidance during the writing process.

The authors also thank Dr. Adriana Arcia from the University of San Diego for her valuable insights on study design, user testing, and visualization evaluation, which greatly informed this work.

Contents

1 Introduction **1**

2 Background **3**

2.1 Reference Range Number Lines . . . 3

2.2 Health Visualizations . . . 4

2.3 Existing Visualization-Based Frameworks and Grammars . . . 5

3 Methods **6**

3.1 Creating a Domain-Specific Language . . . 6

3.1.1 Choosing a Computational Language . . . 6

3.1.2 Identifying Reference Range Number Line Components . . . 7

3.1.3 Number Line Specification Language Properties . . . 7

3.1.4 Capabilities of Creating Number Lines in Vega-Lite . . . 9

3.1.5 Conversion to Vega-Lite Objects . . . 11

3.1.6 Conversion to Vega Objects . . . 12

3.1.7 Gradients in D3 . . . 13

3.2 Replicating a study . . . 14

3.3 Website . . . 17

3.3.1 Graph Editor . . . 18

3.3.2 Additional Packages and Resources . . . 20

3.3.3 Visit the Website . . . 20

4 Results **21**

4.1 Sample Demographics . . . 21

4.2 Perceived test urgency . . . 21

4.3 Behavioral Intentions . . . 24

5 Discussion **26**

5.1 Accessibility and AI Implementation . . . 26

5.2 Reducing Errors and Continued Research . . . 27

6 Limitations **28**

7 Definitions **29**

References **29**

List of Figures

1 Annotated Zikmund-Fisher Platelet Count RRNL[1] 8
2 Properties in our DSL for the replication of Zikmund-Fisher's Platelet Count RRNL[1] 10
3 Zikmund-Fisher Platelet Count RRNL[1] Replicated in Vega-Lite 11
4 Flowchart summarizing the process of our DSL 12
5 Zikmund-Fisher Platelet Count RRNL [1] Replicated in Vega 13
6 Zikmund-Fisher Platelet Count RRNL[1] Replicated in Vega with a gradient added via D3 . . 14
7 Zikmund-Fisher Platelet Count RRNL[1] replication Without HTML Edits 16
8 Zikmund-Fisher Platelet Count RRNL[1] replication with HTML Edits 16
9 Quick Overview 18
10 RRNL Graph Editor on the website 19
11 Options for axis, labels, and separation properties 19
12 Demographics 22
13 Means (standard deviations) of urgency score and urgency difference score ratings, by display format 23
14 Violin plots of urgency difference scores, by display format 25

List of Tables

1 RRNL component classification criteria 9

1 Introduction

The volume and availability of healthcare data have dramatically increased in recent years with the introduction of personal health devices and patient portals[2, 3]. Personal health data can inform patients and guide health-related decisions, but low literacy levels reduce their effectiveness [4]. 21% of Americans have low literacy skills, and 36% of American adults have basic or below basic health literacy, with disadvantaged populations disproportionately making up these groups[5, 6]. The stress of retrieving healthcare information, especially in emergency or intensive care situations, is an additional factor in decision-making. Stressful situations can increase cognitive load and negatively impact information retention [7]. Literacy rates and stressful situations make it difficult to understand, retain, and apply health information toward decision-making, but data visualizations help alleviate this by conveying complex information and reducing cognitive load[8].

Reference range number lines (RRNLs) are visualizations with promising results for increasing comprehension of health information[9, 10, 11, 12]. These visualizations display a single-value datum along a continuous scale that is categorically classified based on reference ranges for that metric. A common example is a number line indicating a blood glucose test result that is low, standard, or high based on its reference ranges [11, 12]. However, RRNLs specifically are not often studied by health researchers. In some cases, researchers study visualizations that share common attributes with reference range number lines, but do not follow the same format, such as horizontal bar charts for lab results or simple number lines for understanding risk[13, 14]. Generalizing findings about RRNLs and similar visualization types is difficult because RRNLs are not well-defined. As a result, there is no current formalized style guide or method for designing RRNLs.

Computational grammars and specification languages can standardize RRNLs by translating their basic components into a framework to support systematic replication[15]. RRNLs are tailored with individualized data and different color schemes, but the basic structure of the visualization stays the same[16]. A grammar allows for the replicability of the basic structure, as well as the expressiveness to tailor the visualization as needed. Grammars also enhance accessibility by helping computers parse visualizations and generate accurate summaries. These summaries can then be translated into other languages or improve accessibility for blind individuals [17]. Exploring current grammars and their support for RRNLs is a vital first step to increasing accessibility and formalizing RRNLs.

In this paper, we analyze the high-level grammar of Vega [18] through the lens of reference range number line replication and create a basic domain-specific language (DSL) for replicating reference range

number lines. The background ties together the current knowledge of RRNLs, health visualization techniques, and existing grammars. Our methods detail our process of translating the properties of RRNLs into a DSL; we then reproduced these RRNLs in Vega and noted its limitations, and used this DSL to replicate an existing RRNL study. Our results summarize our success in replicating this study, while our discussion addresses the benefits of standardizing RRNL creation with a DSL. We conclude with a section noting the limitations of our DSL and future considerations for expanding it, with the ultimate goal of giving RRNLs a space in the visualization community and improving accessibility.

With this project, we propose a domain-specific language that breaks reference range number lines into their core dimensions. Our program provides a straightforward process to generate an RRNL using an object from our DSL. This offers a standardized method to create number lines, replacing the need to build them from scratch each time. Our website showcases our DSL and program, providing information on our DSL and an editor for use of our program.

2 Background

2.1 Reference Range Number Lines

Reference range number lines (RRNLs) are used in high-impact scenarios for patients with chronic diseases[11, 12, 19, 20, 13, 21, 22]. They serve as tools for chronic disease management by demonstrating the patient's current health status based on lab reports and self-reported surveys. For patients with greater barriers to understanding their health status, having access to cognitively accessible graphs makes the difference between good health management and experiencing complications[23]. Reference range number lines have been shown to improve comprehension for individuals with low health literacy and older adults, both of whom have historically higher rates of chronic illness and barriers to comprehension. By exploring how RRNL design variations improve comprehension, researchers can provide structured support that ensures high-risk individuals stay in control of their health.

Existing RRNL studies explore user preference and how design variations influence comprehension, urgency, and self-management behaviors. Users strongly prefer RRNLs compared to other visualization types. For example, Arcia et al. (2019) found that RRNLs were preferred over a stoplight design for Asthma Control Status [9] while Brewer et al. (2012) established a preference for RRNLs over a tabular data format that displays lab results [13]. Further research has been conducted on the specifics of designing comprehensible RRNLs. Arcia et al. (2016) [24] used iterative participatory design to create infographic designs for individuals with low health literacy. They concluded that successful infographics are information-rich and provide context. This finding became a foundation for other studies that measured how additional contextual cues influence user comprehension. One study compared four RRNL designs with increasing contextual cues by adding reference range color, clear reference range titles, and annotations [11]. They found that adding contextual cues does not increase cognitive load and could therefore be included in RRNLs to increase the information richness of a design. Another study found that adding colored anthropomorphic cues, as compared to black-and-white designs with and without anthropomorphic cues, decreased task completion time and increased comprehension of health data [25], further demonstrating the value of contextual cues in RRNLs. Outside of adding contextual cues, other studies have explored the relationship between altering cues and the resulting impacts on user performance. For example, Zikmund-Fisher et al. (2017) [1] explored how variations in color for reference ranges influence perceived urgency and desire to contact health care providers. They found that gradient coloring, as opposed to block coloring, resulted in greater sensitivity to variations in test results. These findings create a baseline for information-rich visualizations that support

health data comprehension and informed decision-making. However, the current methodologies for creating reference range number lines are not systematic, and the computational tools used are often omitted.

2.2 Health Visualizations

Computer visualizations are often used in health to improve risk communication and Bayesian reasoning for medical decision-making. Current visualization tools implement patient-specific visualizations to tailor the decision-making experience. For example, Linhares et al. proposed a visualization tool called ClinicalPath that displays a longitudinal map of a patient's clinical data using symbols and color encoding [26]. This tool displays whether a patient's test result falls within the standard reference range by encoding it with symbols. Other visualizations in ClinicalPath include line charts with test results over time, showing both single tests and how they compare to other tests. ClinicalPath was received positively by participants and produced strong accuracy scores. Another visualization tool, PROACT, was developed to communicate prostate cancer health risk[27]. PROACT employed pie charts, bar graphs, and temporal area charts to showcase different dimensions of risk. Some participants showcased that they were unfamiliar with or had difficulty understanding the temporal area charts. Current computer visualization tools are working to enhance risk perception and continue to inform their design with user feedback, a part of which relies on the ability of a user to understand a graph type.

While bar graphs and pie graphs are often used to communicate risk, icon arrays (also known as pictographs) are a well-studied visualization with a dedicated role in displaying health risk[27, 28, 29]. Icon arrays are graphical representations of the numerators and denominators of a probability using a matrix of icons, such as human figures[28]. When directly compared, these graphics have been shown to outperform other visual forms of risk communication, like bar graphs[30]. They also have the potential to mitigate biases such as denominator neglect, where a person only internalizes the numerator of a probability, ignoring the denominator and leading to a misunderstanding of a percentage[31]. However, the efficacy of icon arrays has been previously determined to be influenced by individual measures. For example, Okan et al. [23] explored how graph literacy impacts the efficacy of icon arrays in overcoming denominator neglect and found that icon arrays more often increased risk comprehension among people with high graph literacy compared to those with low graph literacy. Another study by Ottley et al. [32] measured the impacts of spatial ability on Bayesian reasoning. Ottley et al. tested a variety of text and visualization conditions, including icon arrays, to demonstrate Bayesian reasoning. After analyzing participants' responses, they found that participants with high spatial ability performed significantly better than participants with low spatial ability across all conditions. These studies showcase a small fraction of the work going into health visualization that, when

aggregated, will inform designs that enhance risk perception and patient comprehension.

2.3 Existing Visualization-Based Frameworks and Grammars

The first step we took to address this gap successfully was turning to visualization design problems on reference range number lines that have been approached in other contexts. To do this, we examined domain-specific frameworks that support the creation of complex graphics. Systems such as EnTICE3 and BITSI focus on having streamlined design decisions that enable visual outputs using formalized style guides. This offered a lot of valuable insight into how semantic intent is preserved through user implementations. While these frameworks are not traditional visualization grammars with foundational levels, they showcase the importance of representation of interpretive elements.

One notable example of a visualization framework is EnTICE3 (Electronic Tailored Infographics for Community Engagement, Education, and Empowerment), a system developed to support the generation of customizable infographics from personal health data[33]. EnTICE3 uses a unique style guide as the main communication tool between domain experts and software developers, which ensures that design specifications are recorded consistently and can be interpreted easily. The style guide consists of standardized fields that describe graphical attributes of each infographic. Consistency and clarity across resulting visualizations demonstrate that a structured approach, like EnTICE3's style guide, can communicate design specifications across disciplines, even without prescribed specific implementation details. EnTICE3 also emphasizes semantic transparency in health communication spaces. While EnTICE3 does not use a visualization grammar, the style guides and framework with clear explanations for each serve as a strong example of a well-documented process for creating tailored graphics.

Another structured framework that informed our thinking on visual design contexts was BITSI (Browser-based Infographic Tailoring Self-service Interface), which, similar to EnTICE3, focuses on supporting the creation of health visualizations through well-defined design structures[16]. Even though BITSI isn't fully a visualization grammar, it emphasizes the intentional design of sensitive health information in a clear and direct manner through a formal style guide. This style guide standardized semantic design decisions such as how variables were displayed, color mapping, and text placement. These design decisions remain consistent with designs that were validated by interviewed patients.

Using these ideas within the two frameworks, we transitioned into studying grammar-based visualization systems: a more structured and declarative approach to specifying visual designs. The following structured grammars stood out to us because of their formalized data-driven decisions and creative visual

components. Each of these works illustrates how overlooked specific visual elements can be elevated into bolder constructs within a visualization grammar.

In GoFish: A Grammar of More Graphics!, Pollock and Satyarayan highlight limitations in the traditional Grammar of Graphics, which is the framework that underlies many common visualization tools, such as Vega-Lite. The authors argue that, while Grammar of Graphics has enabled more expressive design beyond fixed chart types, there are still some gaps when attempting to represent less conventional visual forms, which forces designers to rely on custom, low-level graphics. To address these problems, they propose GoFish, a new declarative grammar that uses primitive shapes and graphical operators based on Gestalt to build more complex visualizations [34]. This work illustrates the need to reconsider which visualization types are recognized and taught within visualization literacy frameworks. By prioritizing only familiar charts, educational models risk overlooking number lines and other chart types.

AnnoGram: An Annotative Grammar of Graphics Extension proposes an extension to Wilkinson's Grammar of Graphics that treats annotations as priority components of a visualization over manual overlays, such as drawing on visualizations to make annotations [35]. In many other existing visualization tools, annotations like labels and markers are added using very basic graphic elements. This approach to developing annotations makes it difficult to recycle and reuse elements in different visualizations. What stands out about AnnoGram is its emphasis on semantic intent over simple visual implementation. In the paper, AnnoGram introduces concepts ranging from annotation targets to placement strategies that allow designers to specify what should be annotated and give a solid reasoning, rather than simply a manual encode on how annotations should appear in the given pixel space. This declarative approach aligns closely with our goals regarding our reference range number line DSL, where elements like value indicators and reference ranges will carry valuable semantic meaning that should remain consistent across designs.

3 Methods

3.1 Creating a Domain-Specific Language

3.1.1 Choosing a Computational Language

As data visualization continues to expand in complexity and application, a range of software and libraries have been developed to support their creation, including Vega-Lite and D3. Vega-Lite is a grammar that uses an intuitive JSON syntax to allow the creation and customization of many types of visualizations

[36]. It is based on Vega, a grammar that offers similar functionality but with a greater degree of versatility. D3 is a JavaScript library that is capable of more varied visualizations than Vega-Lite or Vega, but at the cost of complexity [37]. However, none of these libraries include a dedicated method for constructing number lines. Instead, we used their existing capabilities to design RRNLs using built-in features. To start our development of a domain-specific language for creating RRNLs, we chose Vega-Lite as a means of generating our visualizations. By starting with the most concise language, we could build prototype visualizations more efficiently and measure the capabilities and limitations of Vega-Lite to create RRNLs. After identifying some notable limitations, we based a second iteration of our specification on Vega to generate more accurate RRNLs. We then used D3 to add gradients to RRNLs that were not feasible from our understanding of Vega's capabilities.

3.1.2 Identifying Reference Range Number Line Components

To begin our process, we located RRNL examples in the current literature. We searched PubMed, IEEE, and ACM for studies containing RRNL examples to annotate. We used search terms such as " 'reference range' AND 'number line' " or " 'number line' AND 'graph.' " The search terms were iterated on to ensure we found all possible RRNL examples, since there is no standardized way to refer to these graphics. We then searched all applicable papers for images of RRNLs, which we collected for future use.

To begin creating our DSL, we used these RRNL examples to identify common components across RRNLs. By highlighting the components, we can identify common trends in RRNL design to build a baseline structure and identify levels of customization. We annotated our RRNL examples with our own taxonomy that was iterated on as necessary to include the full scope of RRNL design. Figure 1 shows our annotations, and Table 1 shows the finalized taxonomy.

3.1.3 Number Line Specification Language Properties

Using the components identified in our annotated number lines, we created documentation for the required properties for an object in our language. With each component of an RRNL, we considered how that property may be represented in a language and determined any additional properties necessary to support replication of diverse RRNLs. For example, our properties can specify where the value indicator is, its title text, and whether it overlaps the RRNL or points to it. These properties are not exhaustive as to how RRNLs can be constructed, and instead represent the common components found in RRNLs to allow our language to create them with little complexity.

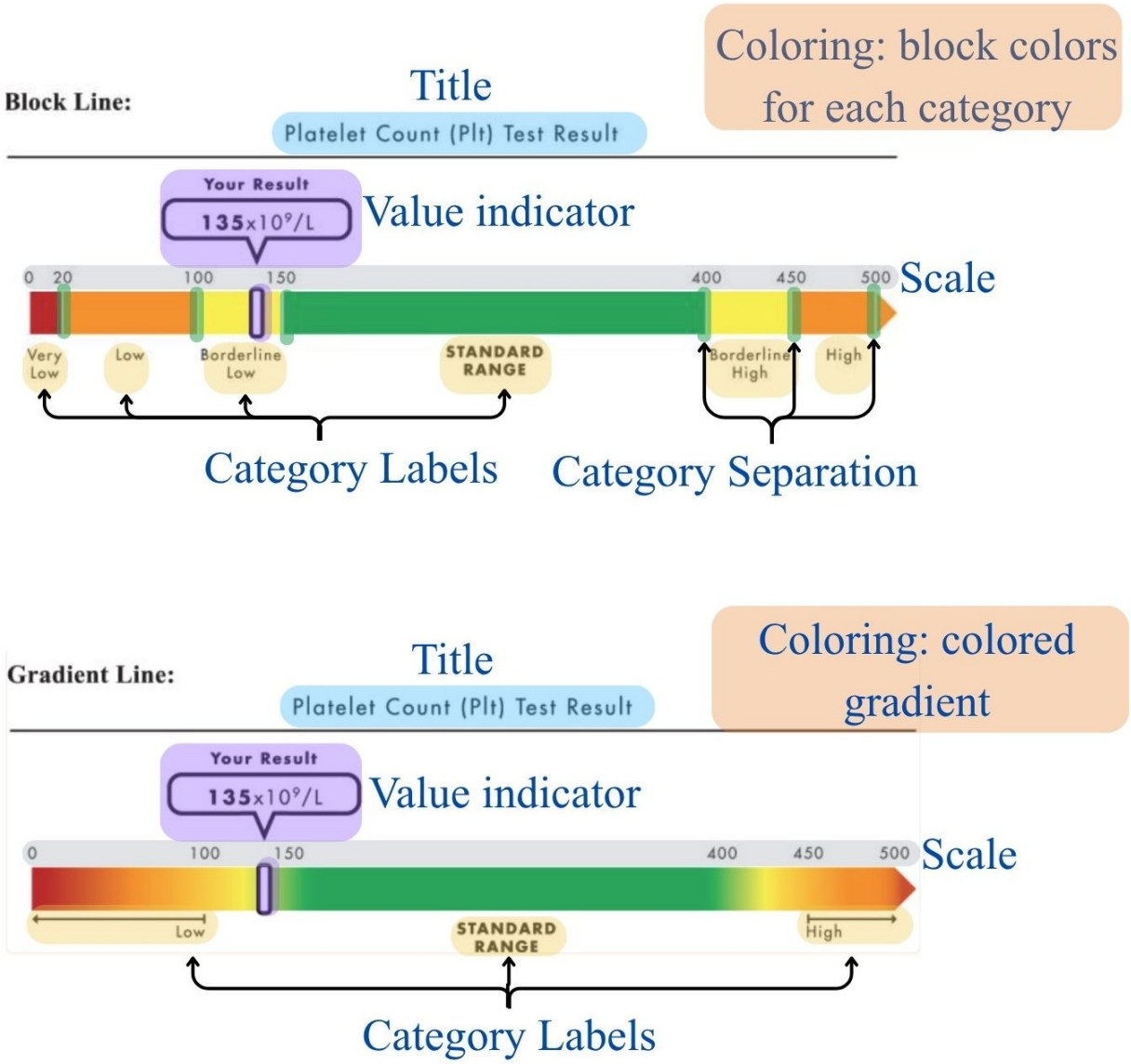

Figure 1: Annotated Zikmund-Fisher Platelet Count RRNL[1]

Component	Requirements
Title	Phrase or sentence introducing or labeling the accompanying visualization. Subtitles or individual labels for each RRNL in double RRNLs were included in this category. Our annotations noted the location of these components.
Coloring	The color scheme used to differentiate reference ranges. It is commonly classified as black-and-white, block, or gradient coloring.
Category Separation	Whether or not the reference ranges and their resulting categories were visually separated on the number line. Our annotations highlight noted where these separators occurred.
Category Labels	The position of the labels for individual categories on the RRNL. Our annotations note whether the labels were on, above, or below the RRNL on the visualization.
Scale	The units of the number lines. Our annotations note the intervals between units and whether the scale was located above or below the number line.
Tick Marks	Consistent symbols that indicate units on the scale. We only considered a number line to have tick marks if there was a line or symbol at a point that clearly indicated a value at that point.
Value Indicator	A visual reference on the number line that points to the specific value being interpreted. Our annotations note if the indicator is above or below the line and whether it overlaps with the number line.
Images/Icons	Any additional visual embellishments. These may contextualize the main idea of the graphic (e.g., an infographic summarizing risks of high blood pressure), or contextualize an individual category (e.g., pictures of different body types for each BMI category).
Additional Text	Extra text that provides context for the graphic.

Table 1: RRNL component classification criteria

We also chose to represent the reference ranges as a list of ordered categories, each of which contains the name of the category, its color on the number line, and the position the reference range ends at. Each category stores only its end value. We accomplished this by creating a "data" property that stores where the first category starts. Each subsequent category immediately starts when the previous one ends. Formatting the language this way prevents users from inputting the same values twice, therefore reducing errors in data entry. With our list of properties, we created several objects, such as the object pictured in Figure 2, using this specification language based on our annotated RRNL examples. Each property was set to match the RRNLs so that one could recreate these number lines as closely as possible given just the objects.

3.1.4 Capabilities of Creating Number Lines in Vega-Lite

While the properties we defined could theoretically be used to construct RRNLs comparable to the examples, we did not yet have a library that could use these objects to recreate RRNLs with the same data, components, and customization. Instead, we started with Vega-Lite to build visualizations that resembled the example number lines as closely as possible. By creating an RRNL in Vega-Lite using the same properties from an object from our specification language, we could examine how viable it would be to use the DSL for generating RRNLs.

Since there is no true number line chart type in Vega-Lite, the best work-around was to build it as a horizontal stacked bar chart with no Y-axis (Figure 3). This would draw only a single stacked bar and create a visualization similar to a number line. The width of the line is extended to mimic a bar, and the X-axis title is removed so that only the number line title remains. Each separate category in the horizontal bar

```
{
  "width": 500,
  "title": {
    "text": "Platelet Count (Plt) Test Result",
    "font": "sans-serif",
    "fontSize": 16,
    "color": "black"
  },
  "data": {
    "categories": [
        {"name": "Very Low", "end": 20, "color": "red"},
        {"name": "Low", "end": 100, "color": "orange"},
        {"name": "Borderline Low", "end": 150, "color": "yellow"},
        {"name": "STANDARD RANGE", "end": 400, "color": "green"},
        {"name": "Borderline High", "end": 450, "color": "yellow"},
        {"name": "High", "end": 500, "color": "orange"}
    ],
    "start": 0
  },
  "labels": {
    "position": "under"
  },
  "tickmarks": {
    "tickCount": 8,
    "position": "top"
  },
  "value_indicator": {
    "value": 135,
    "title": "Your Result: 135x10^9/L",
    "overlap": true
  }
}
```

Figure 2: Properties in our DSL for the replication of Zikmund-Fisher's Platelet Count RRNL[1]

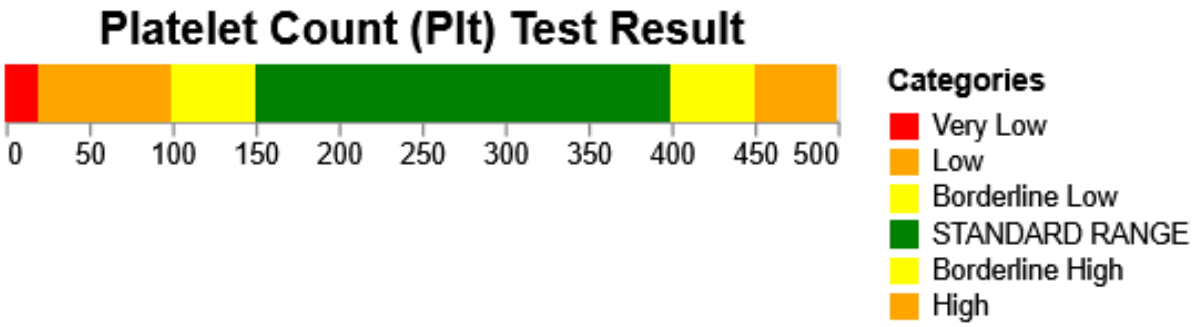

Figure 3: Zikmund-Fisher Platelet Count RRNL[1] Replicated in Vega-Lite

chart becomes its own value in the data with a unique name and color. This way, the graph is recognizable as an RRNL even though it is not technically implemented as one. The title and tick marks were already existing components within Vega-Lite, so they were straightforward to incorporate. To ensure the order and category sizes are correct, each category is given a property based on the range it spans and an additional property to set the order of the categories. By using Vega-Lite's order functionality on this new property, the categories correctly span their intended ranges in the same way as the example RRNL.

However, there are several limitations to creating number lines in Vega-Lite, as some of the components and features of RRNLs are not supported in the DSL. These issues stem from the fact that we were not technically working with number lines, but rather stacked bar charts. From our understanding of Vega-Lite's capabilities, some components of RRNLs that are not typically found in bar charts are difficult or unfeasible to implement in Vega-Lite. The most pertinent example is the value indicator, which is an imperative component of RRNLs that displays to the user precisely where a specific point of data falls on the number line for simple comprehension. Similarly, there is currently no way to include category separators in the number line. Many reference range number lines use gradients instead of block coloring, but Vega-Lite has no standard implementation of adding a gradient to bar charts. Other features of RRNLs could be implemented, but with limits on customization. For example, the category labels of RRNLs are usually positioned underneath or on top of their respective categories, but in Vega-Lite, they are restricted to a legend to the side of the graph. Some RRNLs also only have tick marks at the category separators, but in Vega-Lite, tick marks will always be evenly spaced out.

3.1.5 Conversion to Vega-Lite Objects

To demonstrate that our specification language could be used to construct RRNLs, we created a program that takes in an object representing an RRNL in our specification language and outputs a Vega-

Lite object. We used Python to implement this process, as it is a useful language for automating similar tasks. The program takes in a JSON file that is configured with appropriate properties from our specification language as input. The program then reads each property into a Python object, and these properties are used to construct a dictionary with key value pairs in the same format as a Vega-Lite object. Properties of components that are not supported in Vega-Lite, such as value indicators and category separators, have no impact on the output. When the dictionary is complete, it is converted to a JSON object and output to a file by the program. This file can be opened in Vega-Lite to construct an RRNL with the same data and properties from the original JSON file. Figure 4 summarizes this process.

Figure 4: Flowchart summarizing the process of our DSL

This program allows a user to conveniently create an RRNL in Vega-Lite with their own data. The input files using our specification language take in properties chosen by the creator of the number line, such as the title positioning, the names of categories, and the number of tick marks. Then the program uses these properties to create a readable RRNL in Vega-Lite. Because RRNLs are not officially supported in Vega-Lite, a user might not know how to construct one. It is not immediately apparent that you can imitate one with a stacked bar chart, and figuring out how to position the categories correctly might prove difficult. This program abstracts that part for the user, meaning that they are only responsible for inputting the data and customizations they want. Although not all of the components are supported in this program, it demonstrates the possibility of a more straightforward and convenient way for users to construct RRNLs.

3.1.6 Conversion to Vega Objects

Despite number lines not being officially supported in Vega, Vega's expressiveness helps closely replicate example RRNLs. When creating RRNLs in Vega, the process of creating the base number line is largely the same as it is in Vega-Lite, and the primary changes are the additional components that could not be implemented with Vega-Lite alone.

The two features of Vega that were most useful in creating more accurate RRNLs were the ability

to set a starting and ending X-axis value for each bar and the ability to draw text and symbols onto the visualization with exact coordinates. Each bar of the number line has a field for the start and end values of that bar, meaning the graph can use these values to draw components at the exact positions that they are expected. The mark functionality of Vega allows for symbols, lines, or text to be drawn onto the visualization. They can be set to be drawn at exact positions, meaning that with the right values for the value indicator's text, arrow head, and arrow body, a value indicator can be created in Vega even though it is not an intended Vega feature. The data from the user's object made in our domain-specific language is integrated into the relevant position values, and the final outputted Vega object will draw the category separators, category labels, and value indicator in their correct positions with minimal input from the user.

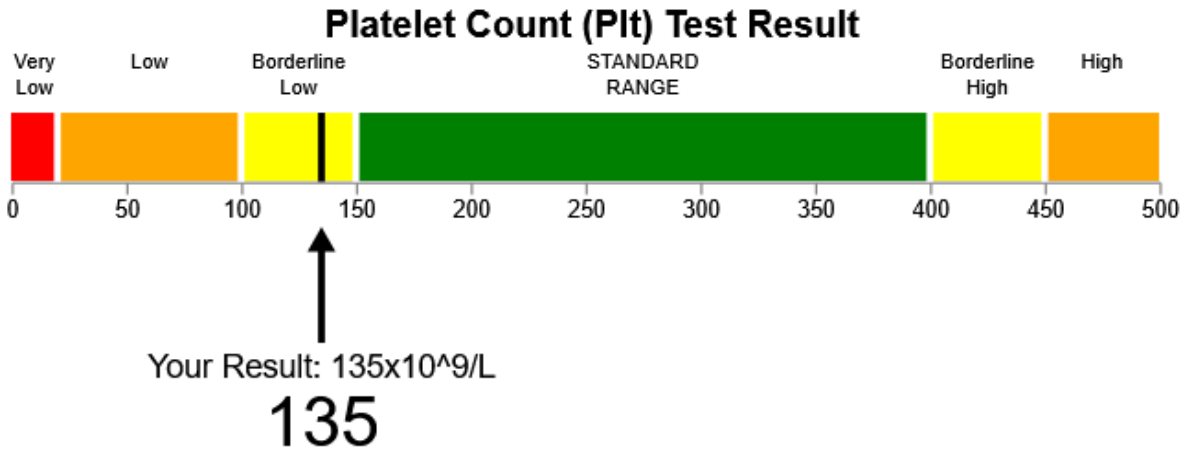

Figure 5: Zikmund-Fisher Platelet Count RRNL [1] Replicated in Vega

Drawing from similarities in the Vega-Lite code, we created a new program that takes a file using our DSL and generates a Vega object that displays an RRNL with the original object's properties (Figure 5). Additionally, it generates an HTML file with the Vega object embedded. The program supports the input of an array of objects in our DSL as well. Each Vega object will be output to its own JSON file, and they will all be present in the outputted HTML file.

3.1.7 Gradients in D3

The only feature that is shared between many RRNLs that Vega did not have a clear solution for is the presence of a gradient across the number line instead of distinct block coloring based on category. Because of the frequency and importance of gradients in perceiving RRNLs[1], we aimed to find an alternate way for users to easily add gradients to their number lines. Due to D3's versatility, we chose to use it as a

library in order to generate number lines with gradients. Since Vega allows exporting objects as SVGs and D3 allows importing SVGs into HTML files, we could use the visualization that was already created in Vega and add a gradient to it.

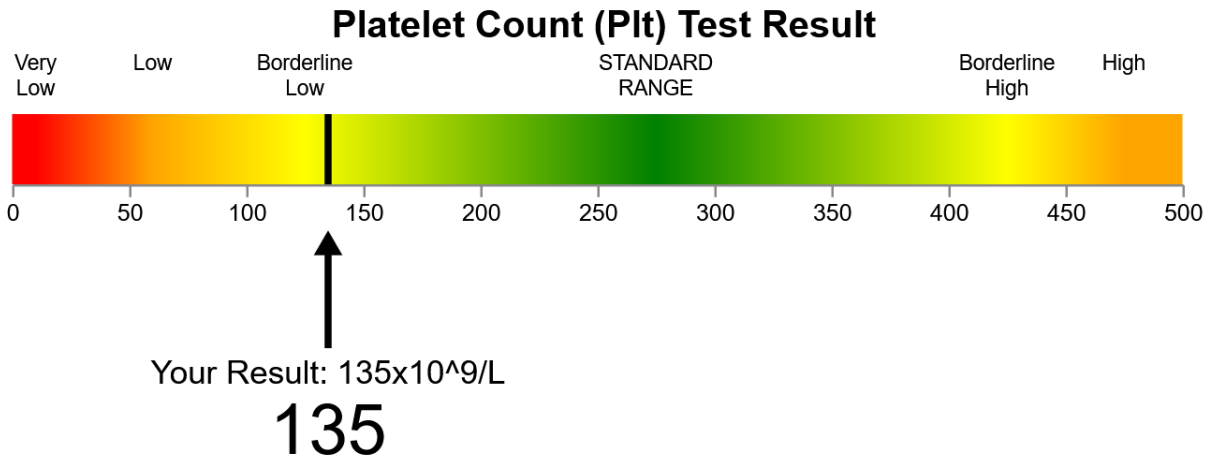

Figure 6: Zikmund-Fisher Platelet Count RRNL[1] Replicated in Vega with a gradient added via D3

The program we created will take in an SVG file created from a Vega object and locate the position and color information of each bar of the number line. It then removes the separate bars that are used to draw the RRNL and replaces them with a single bar of the same size. The information of each bar is used to create a gradient that the new bar is filled with. The resulting page from this file will display the inputted RRNL with a gradient added to the number line. With a combination of our Vega and D3 code, a user can create an RRNL with a gradient by only inputting some necessary parameters into our DSL and running it through our programs.

3.2 Replicating a study

We replicated a reference range number line study with RRNLs generated from our DSL to see how well they reproduced established results. We chose to replicate a Zikmund-Fisher et al. study entitled "Graphics help patients distinguish between urgent and non-urgent deviations in laboratory test results." [1] This study was favorable because we were able to replicate the full scope of the methodology with only small changes to the tools used for participant recruitment and stimuli creation. Additionally, the range of visual stimuli in this study allowed us to find limitations in our DSL.

In the original study from Zikmund-Fisher et al., participants were recruited through Survey Sampling International (SSI) and completed the study on Qualtrics[1]. Our participants were recruited through

Prolific[38] and completed the study on ReVisit. Prolific is an online service that connects participants to studies based on criteria set by the researcher. Participants are financially compensated for their time through Prolific. ReVisit is a study platform designed to help researchers quickly develop and monitor their studies[39].

The original study contained four different visualization types, one of which is a table, while the other three are RRNLs with varying coloring schemes, including a simple gradient, block coloring, and a full gradient. For each visualization type, we created six graphs which vary based on test type (i.e., platelet count, alanine aminotransferase, and serum creatinine levels) and how far the test result was from the standard range (i.e., slightly outside or much further outside of the standard range). We replicated the tables using D3 and replicated the RRNLs with our DSL. We analyzed the original RRNL stimuli and created objects in our DSL with the accompanying information. For RRNLs with block coloring, we generated the RRNL in Vega using its accompanying DSL object and our preestablished Python script. RRNLs with gradient coloring (i.e., simple gradient and full gradient conditions) were generated with Vega code that was then saved to an SVG. This SVG was used as input to our D3 script to match the original study's gradient color scheme. To ensure that our generated HTMLs matched the composition of the original RRNLs, we made some small changes to the HTML that were outside the scope of our DSL. These changes included reversing the direction of the arrow for our value indicator from up to down, adding units to the test result, and selecting where to place our tick marks. Figure 7 shows an example of an RRNL without any changes made to the HTML while Figure 8 demonstrates the final version after editing the resulting HTML file. The final visualizations were displayed to the users on an HTML webpage wrapped with ReVisit. We ensured a similar number of participants were assigned to each condition using the Latin Square feature in ReVisit.

The study methodology was adapted from the original Zikmund-Fisher et al. methodology[1]. The participant was shown six graphs in total. They were shown platelet count (Plt), alanine aminotransferase (ALT), and serum creatinine (SCR) test results in this order. For each test, they were shown a value slightly outside of the standard range, followed by one further outside the standard range. The visualization type for each participant was randomized. For each graph, participants were asked "How alarming does this result feel to you?" and "How urgent of an issue is this result?", each with a 6-point Likert scale where 1 represented "not at all "and 6 represented "very". They were also asked, "Which of the following best describes what you would do in response to your [test name] test result?" with 6 possible answers:

- "Nothing"
- "Talk to your doctor about this test result at your next regular appointment"

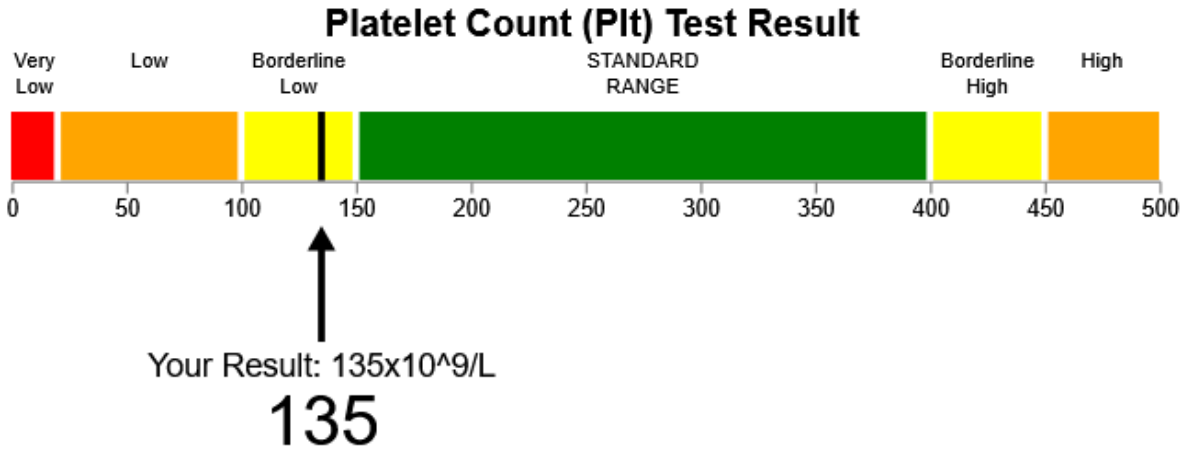

Figure 7: Zikmund-Fisher Platelet Count RRNL[1] replication Without HTML Edits

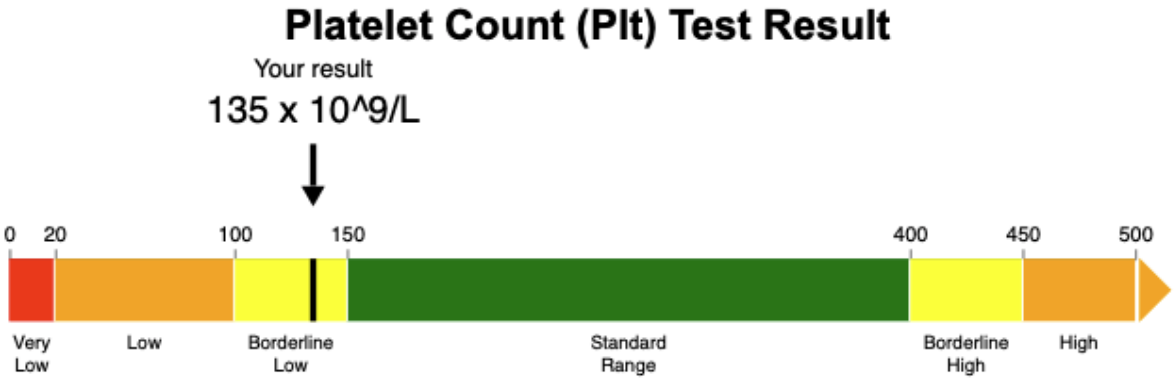

Figure 8: Zikmund-Fisher Platelet Count RRNL[1] replication with HTML Edits

- "Ask to go see your doctor at the first available appointment"
- "Go to a hospital or your doctor's office tomorrow"
- "Go to a hospital as soon as you can get free later today"
- "Go to a hospital immediately"

The participant then completed a section about their preference for the visual stimuli. This section consisted of four questions on 5-point scales. These questions were "In your opinion, how well did these images describe the test results?" on a 5-point scale from "not well at all" to "extremely well," "How helpful were these images in helping you understand the test results?" on a 5-point scale from "not at all helpful to "extremely helpful," "If you were receiving laboratory test results in real life, would you like to see the test results presented using this type of image?" on a 5-point scale from "definitely no" to "definitely yes," and "How much would you trust what these images are telling you about your health?" on a 5-point scale from "do not trust at all" to "trust completely." From there, the participant completed an individual difference measures section, which included demographics and various numeracy and literacy measures. Participants completed the subjective numeracy scale[40], Chew's screening for health literacy[41], questions 5-9, 11, and 13 from Galesic and Garcia-Retamero's graphical literacy scale[42], and then are asked "How familiar are you with the medical test results like the ones discussed in the survey?" on a scale of 1 = "not at all familiar" to 5 = "extremely familiar."

We completed the same data manipulation steps as the original study. This included creating a perceived urgency scale by averaging the results for both questions asked for each graph. It also included calculating an urgency difference score. This score is equal to the perceived urgency of the further value minus the perceived urgency of the value near standard range. Furthermore, we categorized the responses to the behavioral intentions question into two categories: willingness to wait (the first two responses) and taking immediate action (the last four responses). In Python, we then calculated the means and standard deviations for urgency scores and urgency difference scores by display format.

3.3 Website

To present the findings of our project in an accessible and interactive format, we developed a website. The site was built using the Next.js framework. This combination allowed us to create a structured, responsive platform for showcasing both our visualizations and supporting content.

When navigating to the site, users are first brought to the About page. This page provides a concise overview of the project, including the problem being addressed, the proposed solution, and key

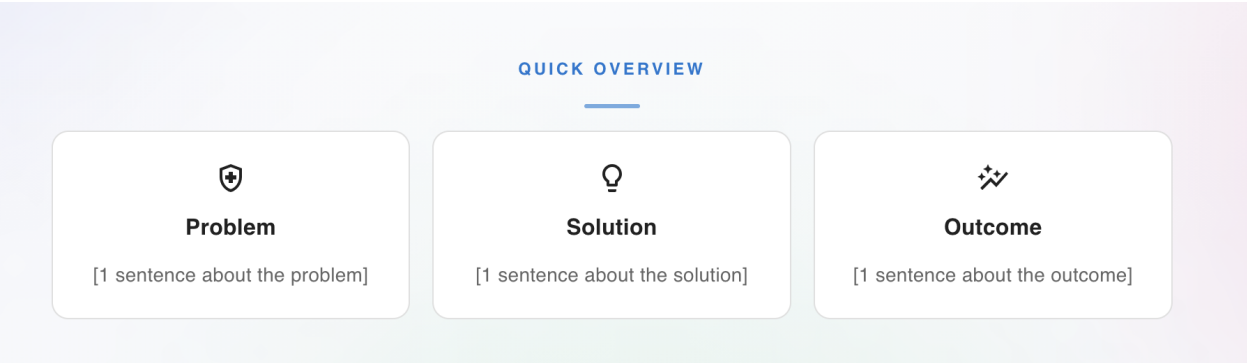

Figure 9: Quick Overview

outcomes, as shown in Figure 9. All significant written content is managed using MDX files, which combine Markdown with JSX. This approach allows team members who were less involved in development to easily update content without modifying the underlying code, ensuring that changes can be made efficiently while maintaining consistency across the site.

Additionally, the website includes a "View the Report" page, which allows users to access and read the final report directly within the site.

3.3.1 Graph Editor

The graph editor is the most substantial and central component of the website. In Figure 10, the graph editor preview window for Vega is shown. The user can switch between a preview of the generated graph and a code window with live-updating code, which can be edited, copied, or downloaded directly.

The interface includes a set of customization controls that are synchronized with the JSON code to demonstrate the capabilities of the DSL. For example, as shown in Figure 11, users can modify axis labels and category separation through these controls. A starting graph can be selected as a base, or users may begin with a default configuration and adjust it using either the property controls or the JSON editor. Additional customizable elements include the value indicator, category definitions, and graph title. Overall, the goal was to expose the full set of properties defined in our DSL through an accessible and user-friendly interface.

Additionally, the website includes a shared graph editor component that provides core functionality used by both the Vega and D3-based visualizations. This component handles tasks such as graph selection, preview rendering, and displaying the underlying code representation (JSON for Vega, HTML/SVG for D3). The Vega editor is primarily used for interactive graph creation and modification, while the D3-based views

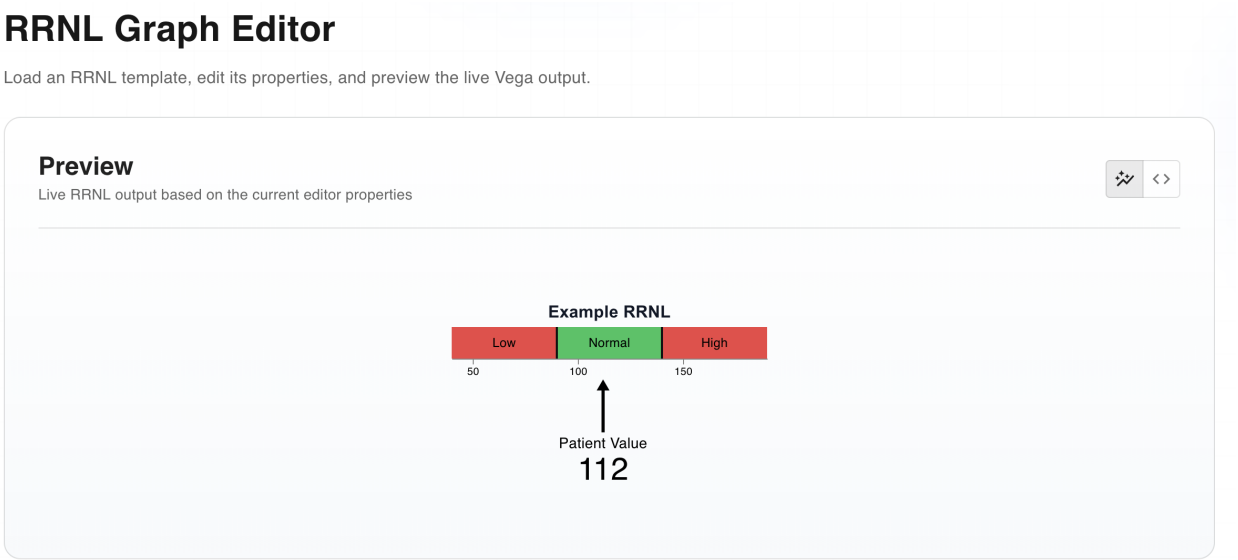

Figure 10: RRNL Graph Editor on the website

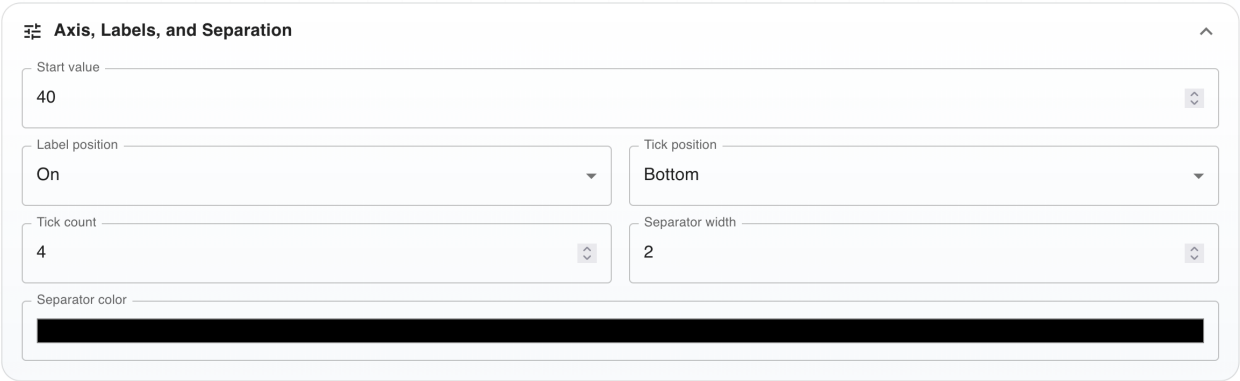

Figure 11: Options for axis, labels, and separation properties

are primarily used to display finalized graphs for the case study. To maintain consistency in the study interface, live editing is disabled on the case study page. However, user-editing can be toggled through the code if desired.

Unlike the original implementation, which used Python scripts to convert specification objects into Vega and Vega-Lite outputs, the website performs this process directly in TypeScript within the client application. The graphs are generated from structured JSON objects based on our domain-specific language, which defines properties such as category ranges, colors, labels, and value indicators. These objects are parsed and transformed into Vega or Vega-Lite specifications in real time, allowing users to immediately see the effects of their changes in the preview window, whether modifications are made through the property controls or directly in the JSON editor.

D3 is used within the case study page to modify rendered SVG outputs and apply more advanced visual features, such as continuous gradients across the number line. All visualizations are generated as HTML files, with some also including an accompanying SVG for additional processing. Further details on this process are provided in Gradients in D3 (3.1.7).

3.3.2 Additional Packages and Resources

Several additional tools and libraries were used to support development and maintain code quality:

- **TypeScript** – Provided static typing to improve reliability and maintainability of the codebase
- **Material UI** – Used for layout, styling, and pre-built UI components across the site
- **Prettier** – Automatically formatted code to ensure consistency across the project
- **Zod** – Used for schema validation when constructing Vega and Vega-Lite objects, ensuring that generated specifications were well-formed and consistent with expected structures
- **Husky** – Enabled pre-commit hooks to enforce code standards and prevent invalid commits

3.3.3 Visit the Website

The final website can be reached at rrnl-showcase.com.

4 Results

To evaluate our DSL-generated RRNLs, we recruited 123 unique participants through Prolific and conducted a replication study of the Zikmund-Fisher et al. study [1] using ReVisit.

4.1 Sample Demographics

123 people completed our study, compared to 1620 from the original study. Figure 12 summarizes the demographics of respondents for our study. Our sample had a mean age of 31.85 years (SD = 10.44). The majority of participants identified as male (54.5%) and as White (56.9%), with Black or African American participants comprising the next largest group (27.6%). Most participants held a Bachelor's degree (57.7%), and 13.8% identified as Hispanic. Health literacy scores were generally high, with a mean of 4.32 (SD = 0.75) on a 1–5 scale, and the mean graphical literacy score was 4.59 (SD = 1.32) out of 7. Mean subjective numeracy was 3.44 (SD = 0.70) on a 1–6 scale, and mean familiarity with medical tests was 3.11 (SD = 1.15) on a 1–5 scale.

4.2 Perceived test urgency

The means and standard deviations of urgency scores and urgency difference scores, broken down by test type and display format, can be found in Figure 13. Across all test types and display formats, mean perceived urgency was higher for extreme values than for near-normal values, which was consistent with the results of the original study from Zikmund-Fisher et al[1].

Across all three test types, RRNL formats outperformed the table format in urgency difference scores. For PLT, the Simple Line format produced the highest difference ($M = 2.53$, $SD = 1.58$) against the table's lowest ($M = 1.97$, $SD = 1.73$). The gap was most pronounced for ALT, where the table scored notably lower ($M = 1.03$, $SD = 0.89$) than all RRNL formats, with Simple Line again highest ($M = 2.69$, $SD = 1.28$). SCR scores were compressed overall, ranging from $M = 0.79$ (Table) to $M = 1.27$ (Blocks Line).

The table format also had the highest rates of zero difference scores, particularly for ALT (29.0%) and SCR (22.6%), indicating it was less effective at helping participants distinguish between near-normal and extreme values. This trend aligns with results from Zikmund-Fisher et al., where the table consistently had higher rates of zero difference scores. However, their zero difference scores for ALT and SCR were 56.3%

Table 1: Demographic & Literacy Numbers

Characteristic	Category	Frequency(%)	Mean(SD)
Age			31.85 (10.44)
Gender	**Male**	67 (54.5%)	
	Female	53 (43.1%)	
	Transgender	0 (0.0%)	
Ethnicity	**Hispanic(any race)**	17 (13.8%)	
	Not Hispanic or Latino	106 (86.2%)	
Race	**White**	106 (86.2%)	
	Black or African American	70 (56.9%)	
	Asian	17 (13.8%)	
	American indian or Alaskan Native	1 (0.8%)	
Education	**High school only**	15 (12.2%)	
	Some college	0 (0.0%)	
	Bachelor's Degree	71 (57.7%)	
	Master's degree	24 (19.5%)	
	Doctoral Degree	0 (0.0%)	
Health Literacy	**1**	1 (0.8%)	4.32 (0.75)
	2	2 (1.6%)	
	3	9 (7.3%)	
	4	56 (45.5%)	
	5	55 (44.7%)	
Subjective Numeracy Scale	**1-1.99**	3 (2.4%)	3.44 (0.70)
	2-2.99	28 (22.8%)	
	3-3.99	62 (50.4%)	
	4-4.99	28 (22.8%)	
	5-5.99	2 (1.6%)	
	6	0 (0.0%)	
Graphical Literacy Scale	**0**	0 (0.0%)	4.59 (1.32)
	1	1 (0.8%)	
	2	11 (8.9%)	
	3	10 (8.1%)	
	4	33 (26.8%)	
	5	33 (26.8%)	
	6	31 (25.2%)	
	7	4 (3.3%)	
Familiarity with Medical Tests	**1**	13 (10.6%)	3.11 (1.15)
	2	25 (20.3%)	
	3	32 (26.0%)	
	4	42 (34.1%)	
	5	11 (8.9%)	

Figure 12: Demographics

Table 2: Means(standard deviations) of urgency score and urgency difference score ratings, by display format

	Near-Normal Test Value	Extreme Test value	Within-Subject Difference Score	Difference Equals 0 (%)
Platelet Count				
Table	2.95 (1.16)	4.92 (1.55)	1.97 (1.73)	6.5
Simple Line	2.87 (1.25)	5.40 (0.84)	2.53 (1.58)	6.5
Blocks Line	2.90 (1.22)	4.95 (0.83)	2.05 (1.39)	3.2
Gradient Line	3.02 (1.21)	5.12 (1.16)	2.10 (1.08)	6.7
ALT				
Table	4.42 (1.36)	5.45 (1.01)	1.03 (0.89)	29.0
Simple Line	2.71 (0.97)	5.40 (0.86)	2.69 (1.28)	6.5
Blocks Line	2.84 (0.97)	5.03 (1.07)	2.19 (1.56)	3.2
Gradient Line	2.78 (1.30)	5.15 (1.25)	2.37 (1.70)	6.7
Serum Creatinine				
Table	3.61 (1.58)	4.40 (1.62)	0.79 (0.83)	22.6
Simple Line	3.21 (1.06)	4.31 (0.97)	1.10 (0.71)	12.9
Blocks Line	3.13 (0.88)	4.40 (0.90)	1.27 (0.74)	3.2
Gradient Line	2.83 (1.10)	3.93 (1.26)	1.10 (0.75)	10.0

Figure 13: Means (standard deviations) of urgency score and urgency difference score ratings, by display format

and 43.7%, respectively [1].

Full distributions are shown in Figure 14. RRNL format distributions were more concentrated above zero than the table, especially for Plt and ALT, while SCR distributions were compressed across all formats — both patterns consistent with the original study.

4.3 Behavioral Intentions

In general, participants were more likely to wait before taking action when results were presented using number line formats rather than tables. This suggests that participants had less urgency when it came to number line formats compared to tables, consistent with the findings of Zikmund-Fisher et al [1].

Overall, our results are broadly consistent with the findings of Zikmund-Fisher et al. [1]. Importantly, these effects were observed using visual stimuli generated from our DSL. This is evidence that our DSL is sufficient for generating effective RRNLs for experimental use.

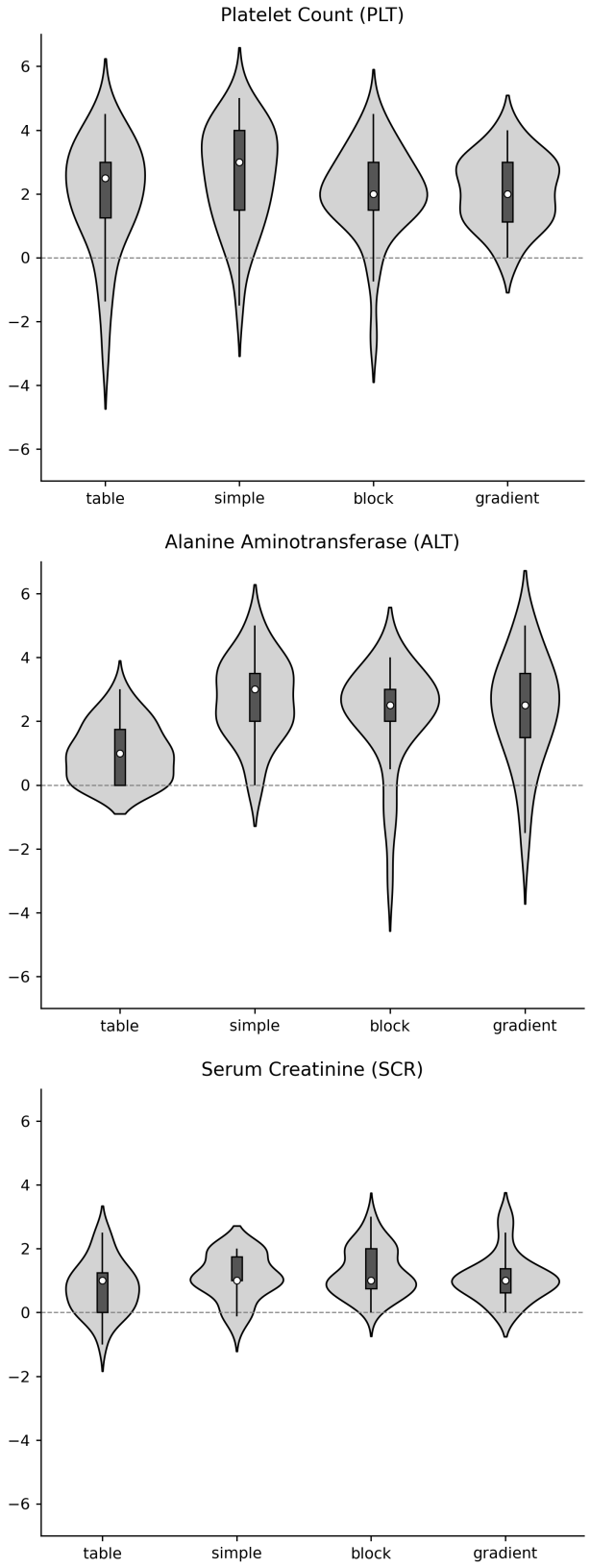

Figure 14: Violin plots of urgency difference scores, by display format

5 Discussion

RRNLs are prevalent in the healthcare landscape but are underrepresented in visualization tools. We propose our DSL as a means to increase RRNL visibility by providing a simplified method for users without extensive computational experience. As a result, our DSL prioritizes usability over extreme customization. Future iterations of this DSL could support greater expressiveness in designing RRNLs.

While creating an RRNL with Vega was inherently complex due to the fact that they are not officially supported, we were able to complete our program by carefully deliberating over what is most necessary for RRNLs and how to reproduce them in Vega. Our final program takes in our straightforward DSL and uses it to create a Vega object using the specified properties. Previously, creating a number line in Vega would require knowledge of JSON formats and indirect manipulation of other existing graph types to visually mimic RRNL formats. Creating an object in our DSL cuts out the time spent manipulating visuals and streamlines the process of making RRNLs.

5.1 Accessibility and AI Implementation

Formalizing a visualization type through a declarative specification works toward our goal of making RRNL creation more accessible. When the structure of a visualization like ours is encoded in a machine-readable language as opposed to a static image, it becomes possible for systems to describe that visualization without human intervention. Our DSL features the semantic components of RRNLs, such as reference range boundaries, category labels, and value indicators, and even named properties rather than as raw graphical primitives.

This structured representation creates an opportunity for screen readers, text-to-speech tools, and other tech to create accurate and consistent natural language descriptions of a patient's lab result directly from the specified object. This approach goes hand in hand with accessibility work done in other visualization domains. For example, efforts to formalize UpSet plots showed that a well-defined DSL standardizes creation and goes even further to enable the generation of textual summaries that can be used by assistive tools. Because the DSL is coded with intent, an automated system does not need to infer what a colored bar represents. It already knows that bar is a reference range category with a defined clinical meaning. RRNLs are used commonly with populations that have lower health and graph literacy, making this kind of output consequential for the exact audiences these visualizations are designed to serve.

5.2 Reducing Errors and Continued Research

The DSL that we developed helps to reduce errors when creating reference range number lines, as it replaces the process of manually constructing them with a structured and reusable specification of the visualization's core components. Rebuilding each number line from scratch risks inconsistencies in category ordering, indicator position, label placement, and range values. Instead, the DSL can be used to create a consistent graph by defining the elements needed through a uniform set of properties.

This DSL is especially valuable in health communication research, as even small changes can threaten the validity of the study. Minor visual differences between graphs can result in drastically different interpretations. As such, we stress the importance of using consistent values and visuals. In addition, the DSL makes it easier for researchers to create graphs more quickly by reducing the barrier to creating RRNL stimuli, regardless of their visualization programming background. By making RRNLs easier and simpler to generate consistently, our DSL can support stronger replication practices and encourage more researchers to conduct studies involving these visualizations.

6 Limitations

While our program is able to generate RRNLs using our DSL, it is not able to create RRNLs with specialized details. Our DSL allows customization to fit a wide variety of general use cases, letting users customize aspects of the graph such as the color of each category, the position of the category labels, and the presence of components like category separators and value indicators. However, some examples of RRNLs have embellishments such as accompanying images for each category or alternate designs for value indicators. While these are possible in Vega and D3, our code's goal is to provide a straightforward method of replicating the vast majority of RRNL examples. Our program's structure could not accommodate all the necessary properties for images, including size, amount, and position, without risking user convenience.

A primary strength of our DSL is how accessible it is for anyone that wants to create an RRNL. While visualization libraries do not always offer an easy way to create an RRNL, our program will generate an RRNL in Vega with little input from the user despite the fact that they are not officially supported in Vega. Since the scales and positions of images would likely differ depending on a user's needs, implementing support for image uploads would be counterintuitive to our DSL's ease of use. Despite these limitations, users who want a visualization that is more specialized can edit the outputted Vega object to add or adjust components in ways that are not possible with our DSL. Many customized components, like value indicator design, would be possible to accomplish in D3 but would require creating code that goes beyond the capabilities of our DSL.

7 Definitions

Domain-specific language (DSL) A computer language that is designed for a specific field or type of application.

Grammar A system consisting of a set of rules that define how to generate valid strings within a language.

Object A self-contained entity that represents an individual item in a domain.

Reference ranges number lines (RRNL) Visualizations common in healthcare contexts which display a single-value datum along a continuous scale that is categorically classified based on reference ranges for that metric.

References

[1] B. J. Zikmund-Fisher, A. M. Scherer, H. O. Witteman, J. B. Solomon, N. L. Exe, B. A. Tarini, and A. Fagerlin, "Graphics help patients distinguish between urgent and non-urgent deviations in laboratory test results," *Journal of the American Medical Informatics Association*, vol. 24, no. 3, pp. 520–528, 2017.

[2] G. Wiederrecht, S. Darwish, and A. Callaway, "The healthcare data explosion." Accessed 2026-01-25.

[3] W. Barker, W. Chang, J. Everson, M. Gabriel, V. Patel, C. Richwine, and C. Strawley, "The evolution of health information technology for enhanced patient-centric care in the united states: Data-driven descriptive study," *Journal of Medical Internet Research*, vol. 26, p. e59791, 2024.

[4] S. K. Smith, A. Dixon, L. Trevana, D. Nutbeam, and K. J. McCaffery, "Exploring patient involvement in healthcare decision making across different education and functional health literacy groups," *Social Science & Medicine*, vol. 69, no. 12, pp. 1805–1812, 2009.

[5] N. C. for Education Statistics, "Adult literacy in the united states," 2019. Accessed 2026-01-26.

[6] M. Kutner, E. Greenberg, Y. Jin, and C. Paulsen, "The health literacy of america's adults: Results from the 2003 national assessment of adult literacy," Tech. Rep. NCES 2006-483, National Center for Education Statistics, Washington, DC, September 2006.

[7] A. J. Porcelli and M. R. Delgado, "Stress and decision making: effects on valuation, learning, and risk-taking," *Current Opinion in Behavioral Sciences*, vol. 14, pp. 33–39, 2017.

[8] S. Park, B. Bekemeier, A. Flaxman, and M. Schultz, "Impact of data visualization on decision-making and its implications for public health practice: a systematic literature review," *Informatics for Health and Social Care*, vol. 47, no. 2, pp. 175–193, 2022.

[9] A. Arcia and M. George, "Reference range number line format preferred by adults for display of asthma control status," *Journal of Asthma*, vol. 57, pp. 638–644, 2019.

[10] A. Arcia, M. George, M. Lor, S. Mangal, and J.-M. Bruzzese, "Design and comprehension testing of tailored asthma control infographics for adults with persistent asthma," *Applied Clinical Informatics*, vol. 10, no. 4, pp. 643–654, 2019.

[11] K. Liu, P. Su, H. Wang, and D. Tao, "Contextualizing visualizations of digital health information among young and older adults based on eye-tracking," *Sustainability*, vol. 14, no. 24, p. 16506, 2022.

[12] D. Morrow, R. F. L. Azevedo, R. Garcia-Retamero, M. Hasegawa-Johnson, T. Huang, W. Schuh, K. Gu, and Y. Zhang, "Contextualizing numeric clinical test results for gist comprehension: Implications for ehr patient portals," *Journal of Experimental Psychology: Applied*, vol. 25, no. 1, pp. 41–61, 2019.

[13] N. T. Brewer, M. B. Gilkey, S. E. Lillie, B. W. Hesse, and S. L. Sheridan, "Tables or bar graphs? presenting test results in electronic medical records," *Medical Decision Making*, vol. 32, no. 4, pp. 545–553, 2012.

[14] C. J. Fitzsimmons, P. G. Sidney, M. K. Mielicki, L. K. Schiller, D. A. Scheibe, J. M. Taber, P. G. Matthews, E. A. Waters, K. G. Coifman, and C. A. Thompson, "Worked examples and number lines improve u.s. adults' understanding of health risks as ratios.," *Journal of Applied Research in Memory and Cognition*, vol. 32, no. 2, pp. 228–248, 2024.

[15] G. Moreira, M. Lage, N. Ferreira, and F. Miranda, "High-level grammars for visualization and visual analytics," in *2023 36th SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*, pp. 270–275, 2023.

[16] A. Arcia, D. Chen, and K. South, "Browser-based infographic tailoring self-service interface (bitsi)," in *2022 Workshop on Visual Analytics in Healthcare (VAHC)*, pp. 01–07, 2022.

[17] T. He, M. McCracken, D. Hajas, S. Creem-Regehr, and A. Lex, "Using tactile charts to support comprehension and learning of complex visualizations for blind and low-vision individuals," 2025.

[18] A. Satyanarayan, K. Wongsuphasawat, and J. Heer, "Declarative interaction design for data visualization," in *Proc. ACM User Interface Software & Technology (UIST)*, 2014.

[19] D. Morrow, M. Hasegawa-Johnson, T. Huang, W. Schuh, R. Ferreira Leitão Azevedo, K. Gu, Y. Zhang, B. Roy, and R. Garcia-Retamero, "A multidisciplinary approach to designing and evaluating electronic medical record portal messages that support patient self-care," *Journal of Biomedical Informatics*, vol. 69, pp. 63–74, 2017.

[20] D. Tao, J. Yuan, and X. Qu, "Presenting self-monitoring test results for consumers: the effects of graphical formats and age," *Journal of the American Medical Informatics Association*, vol. 25, no. 8, pp. 1036–1046, 2018.

[21] A. M. Scherer, H. O. Witterman, J. Solomon, N. L. Exe, A. Fagerlin, and B. J. Zikmund-Fisher, "Improving the understanding of test results by substituting (not adding) goal ranges: Web-based between-subjects experiment," *Journal of Medical Internet Research*, vol. 20, no. 10, p. e11027, 2018.

[22] P. M. Desai, M. Levine, D. J. Albers, and L. Mamykina, "Pictures worth a thousand words: Reflections on visualizing personal blood glucose forecasts for individuals with type 2 diabetes," in *CHI '18 Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, no. 538, pp. 1–13, 2018.

[23] Y. Okan, R. Garcia-Retamero, E. T. Cokely, and A. Maldonado, "Individual differences in graph literacy: Overcoming denominator neglect in risk comprehension," *Journal of Behavioral Decision Making*, vol. 25, no. 4, pp. 390–401, 2011.

[24] A. Arcia, N. Suero-Tejeda, M. E. Bales, J. A. Merrill, S. Yoon, J. Woollen, and S. Bakken, "Sometimes more is more: iterative participatory design of infographics for engagement of community members with varying levels of health literacy," *Journal of the American Medical Informatics Association*, vol. 23, no. 1, pp. 174–183, 2016.

[25] P. Su and K. Liu, "Effects of anthropomorphic design on comprehension of self-monitoring test results: Integrating evidence of eye-tracking and event-related potential," *Displays*, vol. 82, p. 102664, 2024.

[26] C. D. G. Linhares, D. M. Lima, J. R. Ponciano, M. M. Olivatto, M. A. Gutierrez, J. Poco, C. Traina, and A. J. M. Traina, "Clinicalpath: A visualization tool to improve the evaluation of electronic health records in clinical decision-making," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 10, pp. 4031–4046, 2023.

[27] A. Hakone, L. Harrison, A. Ottley, N. Winters, C. Gutheil, P. K. J. Han, and R. Chang, "Proact: Iterative design of a patient-centered visualization for effective prostate cancer health risk communication," *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 1, pp. 601–610, 2017.

[28] J. S. Ancker, N. C. Benda, M. M. Sharma, S. B. Johnson, S. Weiner, and B. J. Zikmund-Fisher, "Taxonomies for synthesizing the evidence on communicating numbers in health: Goals, format, and structure," *Risk Analysis*, vol. 42, no. 12, pp. 2656–2670, 2022.

[29] J. S. Ancker, Y. Senathirajah, R. Kukafka, and J. B. Starren, "Design features of graphs in health risk communication: A systematic review," *Journal of the American Medical Informatics Association*, vol. 13, no. 6, pp. 608–618, 2006.

[30] J. Kandel, J. Liu, A. Z. Wang, C. Tseng, and D. A. Szafir, "Graphical perception of icon arrays versus bar charts for value comparisons in health risk communication," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 1, pp. 780–790, 2026.

[31] R. Garcia-Retamero and M. Galesic, "Communicating treatment risk reduction to people with low numeracy skills: A cross-cultural comparison," *American Journal of Public Health*, vol. 99, pp. 2196–2202, 2011.

[32] A. Ottley, E. M. Peck, L. T. Harrison, D. Afergan, C. Ziemkiewicz, H. A. Taylor, P. K. J. Han, and R. Chang, "Improving bayesian reasoning: The effects of phrasing, visualization, and spatial ability," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 1, pp. 529–538, 2016.

[33] A. Arcia, M. Velez, and S. Bakken, "Style guide: An interdisciplinary communication tool to support the process of generating tailored infographics from electronic health data using entice3," *eGEMs (Generating Evidence & Methods to improve patient outcomes)*, vol. 3, no. 1, p. 3, 2015.

[34] J. Pollock and A. Satyanarayan, "Gofish: A grammar of more graphics!," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 1, pp. 549–559, 2026.

[35] M. D. Rahman, M. R.-u. Zaman, A. M. McNutt, and P. Rosen, "Annogram: An annotative grammar of graphics extension," *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[36] Trifacta, "Vega: A visualization grammar," 2014. Accessed: 2026-04-15.

[37] M. Bostock, V. Ogievetsky, and J. Heer, "D3 data-driven documents," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, p. 2301–2309, Dec. 2011.

[38] "About prolific." Accessed 2026-02-26.

[39] Y. Ding, J. Wilburn, H. Shrestha, A. Ndlovu, K. Gadhave, C. Nobre, A. Lex, and L. Harrison, "revisit: Supporting scalable evaluation of interactive visualizations," in *2023 IEEE Visualization and Visual Analytics (VIS)*, pp. 31–35, 2023.

[40] A. Fagerlin, B. J. Zikmund-Fisher, P. A. Ubel, A. Jankovic, H. A. Derry, and D. M. Smith, "Measuring numeracy without a math test: Development of the subjective numeracy scale," *Medical Decision Making*, vol. 27, no. 5, pp. 672–680, 2007.

[41] L. D. Chew, K. A. Bradley, and E. J. Boyko, "Brief questions to identify patients with inadequate health literacy," *Family Medicine*, vol. 36, no. 8, pp. 588–596, 2004.

[42] M. Galesic and R. Garcia-Retamero, "Graph literacy: A cross-cultural comparison," *Medical Decision Making*, vol. 31, no. 3, pp. 444–457, 2010.